

Q.N. ① Restaurant Information System - DFD

Q.N. ① Apex Global Hospital - ER Diagram

⇒ Conceptual ER Design (Crow's Foot Notation):

The hospital has many Doctors and Patients. Since a Doctor can treat many Patients and a Patient can be treated by many Doctors, this forms a many-to-many (M:N) relationship. To resolve this, we use an Associative Entity called **Consultation**.

Entities & Attributes:

- **Doctor** : DoctorID (PK), Name, Specialization, Phone, DeptID (FK)
- **Patient** : PatientID (PK), Name, DOB, Gender, ContactNo
- **Consultation** : ConsultID (PK), DoctorID (FK), PatientID (FK), Date, Diagnosis
- **Department** : DeptID (PK), DeptName, Location, HeadDoctorID (FK)
- **Room** : RoomID (PK), RoomType, DeptID (FK), Capacity

ER Diagram (Crow's Foot Notation):

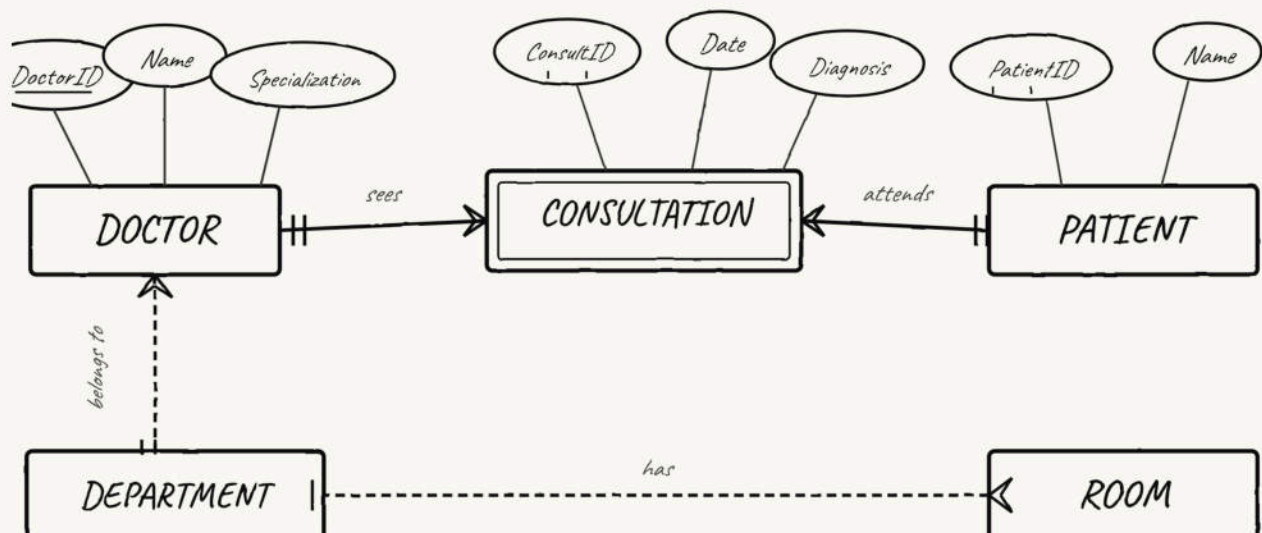


Fig: ER Diagram for Apex Global Hospital (Crow's Foot)

→ The double-bordered rectangle for **Consultation** shows it is an Associative Entity.
The crow's foot (many) is on the Consultation side, resolving the M:N between Doctor and Patient into two 1:M relationships.

Name: Aakash Bhusal Roll no: 822405002

Q.N. ② HealthLink Database – Six Phases of DBLC

⇒ (a) Requirements

⊙ Phase 1 – Requirements Collection & Analysis

- Manage Patient records (demographics, medical history)
- Real-time Video Consultation logs between rural clinics & urban doctors
- Pharmacy Prescription management linked to consultations
- Network of "Health Posts" across provinces with connectivity status
- High concurrency during peak hours
- Patient confidentiality (encryption + RBAC)
- Offline-first support for rural clinics (intermittent internet)

Functional Requirements:

- User Registration & Login (Doctor, Patient, Admin, MoH)
- Schedule & Manage video consultations
- Patient Record Creation & Maintenance
- Prescription Generation by Doctor
- Central Server Data Storage & Retrieval
- Report Generation for MoH Admin
- Sync Management for rural Health Posts

Non-Functional Requirements:

- *Security: Patient data must be encrypted & role-based access control*
 - *Performance: Video latency <150ms*
 - *Availability: Upto 99.9% uptime*
 - *Scalability: Handle 500+ concurrent video sessions*
 - *Usability: Simple UI, accessible in rural areas*
 - *Reliability: Auto backup every 24 hrs.*
-

⊙ Phase 2 – Conceptual Database Design

Main Entities identified:

- *Patient (PatientID, Name, DOB, Province, HealthPostID)*
- *HealthPost (PostID, PostName, Province, ConnectivityLevel)*
- *Doctor (DoctorID, Name, Specialization, HospitalID)*
- *VideoConsultation (ConsultID, PatientID, DoctorID, PostID, DateTime, Duration, Notes)*
- *Prescription (PrescID, ConsultID, PatientID, MedicineList, IssuedDate)*
- *Hospital (HospitalID, HospName, City, Type: urban/district)*

Key Relationships:

- *Patient belongs to one HealthPost (M:1)*
- *VideoConsultation is Associative Entity for Doctor ↔ Patient (M:N resolved)*
- *Each Prescription is linked to exactly one Consultation (1:1)*

⊙ DFD Diagram for HealthLink System

DFD Level-0 (Context Diagram):

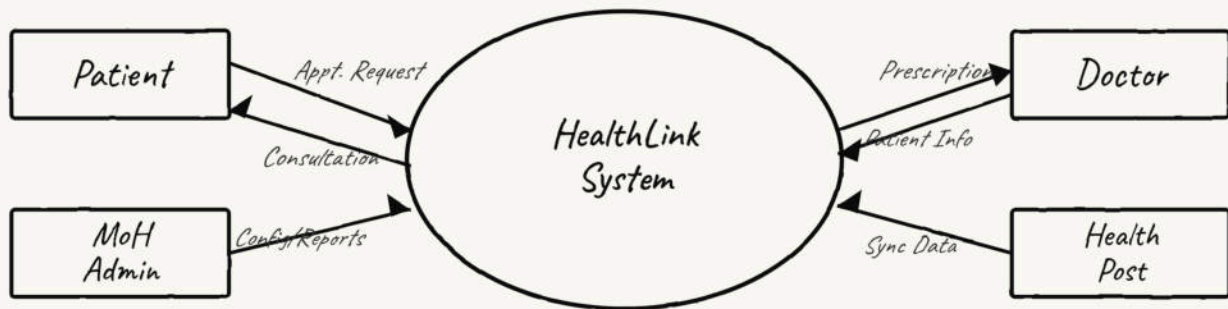


fig: DFD Level-0 for HealthLink System

⊙ Phase 3 – Logical Database Design

Tables (Relational Schema):

- Patient(PatientID, FullName, DOB, Gender, Phone, PostID [FK])
- HealthPost(PostID, PostName, Province, ConnType, IsActive)
- Doctor(DoctorID, FullName, Specialty, HospitalID [FK], Email)
- Hospital(HospitalID, HospName, City, ContactNo)
- VideoConsultation(ConsultID, PatientID [FK], DoctorID [FK], PostID [FK], DateTime, Duration_mins, Notes, Status)
- Prescription(PrescID, ConsultID [FK], PatientID [FK], MedDetails, DosageInfo, IssuedOn, ValidTill)
- SyncLog(LogID, PostID [FK], SyncTime, RecordsSynced, SyncStatus)

All tables are in 3NF. No partial or transitive dependencies exist.

⦿ Phase 4 – Physical Database Design

DBMS Choice:

- Urban Hospitals: *PostgreSQL* (ACID compliance, high concurrency via MVCC)
- Rural Health Posts: *SQLite* (offline-first, syncs to central DB when internet available)

Indexes Created:

- Index on *Patient.PostID* (frequent join)
- Index on *VideoConsultation.DateTime* (date range queries)
- Composite Index on *VideoConsultation (DoctorID + PatientID)*
- Index on *Prescription.ConsultID* (FK join)

Security:

- Encrypt sensitive columns (Name, DOB, Phone) using AES-256
- Role-Based Access Control (RBAC) – Pharmacist sees only prescriptions
- Partitioning on *VideoConsultation* by Year (Range partitioning on *DateTime*)

⦿ Phase 5 – Database Implementation

Sample SQL:

```
CREATE TABLE Patient (  
    PatientID SERIAL PRIMARY KEY,  
    FullName VARCHAR(100) NOT NULL,  
    DOB DATE NOT NULL,  
    PostID INT REFERENCES HealthPost(PostID)  
);
```

- Triggers for audit logging (who updated what and when)
 - Stored procedures for "get all pending consultations for a doctor"
 - Optimistic locking to prevent double-booking a doctor's time slot
 - Load tested with 500+ concurrent connections – all queries under 200ms
-

⊙ Phase 6 – Database Maintenance & Monitoring

- Backups: Full backup daily, incremental every hour (urban hospitals)
- Rural posts sync + backup whenever internet is available
- Performance tuning: Monitor slow queries via `pg_stat_statements`
- Data archiving: Move consultation records older than 5 years to archive schema
- Security audits: Review access logs monthly for unusual patterns
- Sync conflict resolution: Timestamp-based "last write wins" + admin alerts for critical conflicts

Roll no: 822405002

Q.N. ③ Hospital Management System – Case Study (CASE Tools)

⇒ 1) Upper-CASE vs Lower-CASE Activities

Upper-CASE Tools

- ⊙ Requirement collections & analysis
- ⊙ Preparing system models (DFD, ERD)
- ⊙ Drawing flowcharts
- ⊙ Creating UML Diagrams
- ⊙ Database schema Design

Lower-CASE Tools

- ⊙ Writing/coding source code
- ⊙ Unit testing & integration testing
- ⊙ Debugging errors
- ⊙ Version Control & CI/CD
- ⊙ Deploying the system to hospital server

⇒ 2) Two Upper-CASE Tools for HMS

a) Enterprise Architect (Sparx Systems)

- Used for UML modeling, use case diagrams & so on.

b) Microsoft Visio / Rational Rose

- Used for DFDs, system architecture diagrams.

⇒ 3) Two Lower-CASE Tools for HMS

a) IntelliJ IDEA / Eclipse

→ Integrated Development Environment (IDE) used for writing Java/Python code.

b) JUnit / Selenium

→ Testing framework used for unit testing & automated testing.

⇒ 4) Importance of Upper-CASE Tools Before Coding:

- ⊙ Clear Blueprint: Diagrams give developer a clear roadmap before writing code.
 - ⊙ Reduces Rework: Design flaws caught early, cost less to fix than post-coding errors.
 - ⊙ Stakeholder Communication: Visual models help non-technical users validate requirements.
 - ⊙ Better Architecture: Proper design ensures the system is scalable and maintainable.
 - ⊙ Documentation: UML/DFD diagrams serve as official documentation for system lifecycle.
-

⇒ 5) Issues if Coding Starts Without Proper Design:

- ⊙ Incomplete Requirements: Developer may miss key features like billing integration or doctor scheduling.
- ⊙ Poor System Architecture: Without design, the codebase becomes tightly coupled and hard to maintain.
- ⊙ High cost of changes: Fixing design flaws discovered during/after coding is 10x more expensive.
- ⊙ Security Vulnerabilities: Lack of security planning leads to data breaches.
- ⊙ Project Failure: May completely fail to meet user needs.

